

Unit – 3 : Session – 6 : SLO – 2 | Priority and Round Robin Programs — Missing Code (Round Robin)

Answers

BLANK 1: tq (if the remaining time fits in one quantum, the process finishes in this turn).

BLANK 2: tq (otherwise the CPU time advances by one full quantum).

BLANK 3: turnaround - bt[i] (waiting time = turnaround time - burst time, i.e. time - at[i] - bt[i]).

Completed program

```
#include <stdio.h>
int main() {
    int i, n, time = 0, bt[10], rt[10], at[10], tq, remain, flag = 0;
    int wt = 0, tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    remain = n;
    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for Process %d: ", i + 1);
        scanf("%d%d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
    printf("\nProcess\tTurnaround Time\tWaiting Time\n");
    for(time = 0, i = 0; remain != 0;) {
        if(rt[i] > 0 && at[i] <= time) {
            if(rt[i] <= tq) {
                // BLANK 1
                time += rt[i];
                rt[i] = 0;
                flag = 1;
            }
            else {
                rt[i] -= tq;
                time += tq;
                // BLANK 2
            }
            if(flag == 1) {
                remain--;
                int turnaround = time - at[i];
                int waiting = turnaround - bt[i]; // BLANK 3
                printf("P%d\t%d\t%d\n", i + 1, turnaround, waiting);
                wt += waiting;
                tat += turnaround;
                flag = 0;
            }
        }
        if(i == n - 1)
            i = 0;
        else if(at[i + 1] <= time)
            i++;
        else
            i = 0;
    }
    printf("\nAverage Waiting Time = %.2f", (float)wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", (float)tat / n);
}
```

```
    return 0;  
}
```